

STANDARD OPERATING PROCEDURE

Duton / Florosense Development Runbook

How to start the stack, follow each workflow, and read the code that runs it.

Document ID	SOP-DUTON-DEV-001
Revision	1.0
Effective date	08 September 2026
Audience	Developers and operators

1. Document control

Field	Value
Document ID	SOP-DUTON-DEV-001
Title	Duton / Florosense Development Runbook
Revision	1.0
Date	08 September 2026
Product	Duton operator dashboard + Florosense device API
Code folders	duton-frontend (Next.js :3000), duton-backend (Express :8001)
Related docs	docs/INDEX.md and numbered guides 01–05

This SOP is a runbook. Follow the numbered steps in order the first time you set up a machine. After that, jump to the procedure you need (start, login, live sensor, ticket, shutdown). Each procedure shows the command or click, the source file that runs, a short code extract, and why that code exists.

2. Purpose and scope

Purpose: give anyone on the team a single file they can open and use to run Duton locally, trace a user action from the browser into Express and MongoDB, and understand what each piece of code is for.

Scope: local development of duton-frontend and duton-backend. Production deploy (PM2, Nginx) is out of scope except where noted. Device firmware is out of scope; live numbers come from the Florosense cloud API.

3. Definitions

Term	Meaning in this project
Frontend	Next.js app in duton-frontend. What you see at http://localhost:3000
Backend	Express API in duton-backend. JSON at http://127.0.0.1:8001
JWT	JSON Web Token stored in localStorage as <code>duton_access_token</code> after login
Sensor	One air monitor. Master row in MongoDB collection <code>sensors</code>
Reading	One sample of PM2.5, PM10, temperature, humidity in <code>sensor_readings</code>
Calibration	Formula $k_0 + k_1 * \text{raw} + \text{variation}$ applied to displayed PM
AMC	Annual Maintenance Contract / warranty tracking per sensor
Magic link	Guest JWT for one sensor, opened at <code>/access?t=&s=</code>
RBAC	Role-based access: admin, assignee, user, builder, contractor, guest

4. Prerequisites (before any procedure)

- Node.js 18 or newer (this repo was documented with Node 24).
- A MongoDB connection string (local or Atlas). Backend will not start without `NEW_DB_URL`.
- Florosense base URL and API key. Backend will not start without them.
- Two terminals: one for duton-backend, one for duton-frontend.

5. Procedure A — First-time setup

A.1 Create backend environment file

Step A.1. In duton-backend, copy .env.example to .env and fill required values.

Expected result: A .env file exists next to package.json. Do not commit secrets.

```
cp .env.example .env      # Git Bash / macOS / Linux
copy .env.example .env    # Windows Command Prompt
Copy-Item .env.example .env # PowerShell
```

Location: duton-backend/.env (loaded by src/config.js, not by the frontend).

```
NEW_DB_URL=mongodb://username:password@localhost:27017/duton
TICKET_DB_NAME=duton
JWT_SECRET=change-this-to-a-long-random-string
FLOROSENSE_BASE_URL=https://your-florosense-host
FLOROSENSE_API_KEY=your-key
TICKET_HOST=127.0.0.1
TICKET_PORT=8001
```

Minimum keys the backend requires to boot.

Purpose of this code: config.js reads this file at process start. NEW_DB_URL, FLOROSENSE_BASE_URL and FLOROSENSE_API_KEY are mandatory. If they are missing, getSettings() throws and server.js never listens.

A.2 Install dependencies

Step A.2. Install npm packages in BOTH folders. They are two separate Node projects.

Expected result: Each folder has a node_modules directory. No error from npm install.

```
cd duton-backend
npm install

cd ../duton-frontend
npm install
```

package.json in each folder lists the libraries that folder needs.

Purpose of this code: Backend package.json pulls Express, MongoDB driver, JWT, bcrypt, node-cron, SendGrid, nodemailer. Frontend package.json pulls Next.js, React, Tailwind/shadcn, Leaflet, Recharts. Installing only one folder is not enough — login will fail if the API is missing, and the API alone has no UI.

A.3 Optional frontend env

Step A.3. If the API is not on localhost:8001, create duton-frontend/.env.local.

Expected result: Next.js picks up NEXT_PUBLIC_* values at dev-server start.

```
NEXT_PUBLIC_TICKET_API_URL=http://localhost:8001/api
NEXT_PUBLIC_BACKEND_URL=http://localhost:8001
BACKEND_URL=http://localhost:8001
```

Login form and utils/api.js use TICKET_API_URL. next.config.mjs uses BACKEND_URL for rewrites.

6. Procedure B — Start the backend

Step B.1. From duton-backend run npm run dev.

Expected result: Console prints Connected to MongoDB and Local: http://127.0.0.1:8001

```
cd duton-backend
npm run dev
```

package.json script: node --watch src/server.js (restarts when you save a backend file).

What that command actually starts

npm run dev launches src/server.js. That file is the process entry point. It loads settings, creates the Express app, mounts every /api router, connects MongoDB, starts three background jobs, then listens.

```
import { getSettings } from "./config.js";
import * as database from "./services/database.js";
import ticketsRouter from "./routes/tickets.js";
import adminRouter from "./routes/admin.js";
import usersRouter from "./routes/users.js";
import sensorsRouter from "./routes/sensors.js";
// ... assignee, sites, calibration, alerts, amc

const settings = getSettings();
const app = express();
app.use(helmet());
app.use(cors(corsOptions));
app.use(express.json());

app.use("/api/tickets", ticketsRouter);
app.use("/api/admin", adminRouter);
app.use("/api/users", usersRouter);
app.use("/api/sensors", sensorsRouter);
app.use("/api/assignee", assigneeRouter);
app.use("/api/sites", sitesRouter);
app.use("/api/admin/calibration", calibrationRouter);
app.use("/api/alerts", alertsRouter);
app.use("/api/amc", amcRouter);

app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```

File: duton-backend/src/server.js

Purpose of this code: helmet sets security headers. cors lets the browser on port 3000 call port 8001. express.json() parses POST bodies. Each app.use(/api/...) is how a URL becomes a feature. GET /health is the liveness check used in Procedure D — it does not need a login.

After listen: database and jobs

```
async function start() {
  await database.connect();
  await database.initializeDefaultAdmin();
  startSensorReadingCollector(); // every 10 minutes
  startTicketAssignmentScheduler(); // 10:00 and 18:00 IST
  startAlertMonitor(); // every 15 minutes
  app.listen(settings.PORT, settings.HOST, () => { /* log URLs */ });
}
start();
```

File: duton-backend/src/server.js (start())

Purpose of this code: connect() opens MongoDB using NEW_DB_URL. initializeDefaultAdmin() creates the first admin if the admins collection is empty. The three start*() calls keep working even if nobody has the website open: they poll devices, email assignees, and send offline alerts.

7. Procedure C — Start the frontend

Step C.1. In a second terminal, from duton-frontend run `npm run dev`.

Expected result: Console prints Local: `http://localhost:3000`. Keep the backend terminal running.

```
cd duton-frontend
npm run dev
```

package.json script: next dev — Next.js App Router, default port 3000.

First files Next.js loads

```
export const metadata = {
  title: "Duton",
  description: "Duton is a platform for monitoring and managing your sensors.",
};

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={roboto.variable}>
      <body>
        <ThemeProvider defaultTheme="dark" enableSystem>
          {children}
          <ToasterProvider />
        </ThemeProvider>
      </body>
    </html>
  );
}
```

File: duton-frontend/src/app/layout.js — wraps EVERY page.

Purpose of this code: This is the UI entry wrapper. ThemeProvider turns on dark mode by default. ToasterProvider (sonner) is how success/error toasts appear. Roboto is the app font. You do not navigate to this file; Next.js injects it around `/login`, `/dashboard`, and the rest.

```
import { redirect } from "next/navigation"

export default function Home() {
  redirect("/login")
}
```

File: duton-frontend/src/app/page.js — URL /

Purpose of this code: Opening `http://localhost:3000` never shows a home marketing page. It immediately sends the browser to `/login` so the first human step is always authentication.

8. Procedure D — Prove both processes are up

Step D.1. In a browser or curl, open the backend health URL.

Expected result: JSON body: `{ "status": "ok" }`

```
curl http://127.0.0.1:8001/health
# or paste the URL in the browser
```

Step D.2. Open `http://localhost:3000`

Expected result: Address bar becomes `/login` and you see the Duton login screen (video background + form).

If you see	Likely cause	What to do
Cannot GET <code>/health</code>	Backend not running or wrong port	Repeat Procedure B. Check <code>TICKET_PORT</code> in <code>.env</code>

If you see	Likely cause	What to do
NEW_DB_URL must be set	Missing Mongo string	Fix duton-backend/.env and restart npm run dev
FLOROSENSE_BASE_URL must be set	Missing device API config	Set FLOROSENSE_BASE_URL and FLOROSENSE_API_KEY
Site cannot be reached on :3000	Frontend not running	Repeat Procedure C
Login page but API errors later	CORS or TICKET_API_URL mismatch	Confirm frontend calls :8001

9. Procedure E — Login (run this workflow)

Who: any role. Where: /login. Code: LoginScreen → LoginForm. Backend: three possible URLs depending on who you are.

Step E.1. On /login type username and password. Click sign in.

Expected result: On success the browser goes to /dashboard. On failure a red error stays on the form.

Code that chooses the login URL

```
const trimmedUsername = username.trim()
const isAdminLogin = trimmedUsername.toLowerCase() === "admin"

if (isAdminLogin) {
  loginUrl = `${TICKET_API_URL}/admin/login` // POST /api/admin/login
} else {
  loginUrl = `${TICKET_API_URL}/assignee/login` // POST /api/assignee/login
}

let response = await fetch(loginUrl, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ username: trimmedUsername, password }),
})

// If assignee login failed, try regular user login
if (!response.ok && !isAdminLogin) {
  loginUrl = `${TICKET_API_URL}/users/login` // POST /api/users/login
  response = await fetch(loginUrl, { /* same body */ })
}
```

File: duton-frontend/src/components/auth/login-form.jsx (handleSubmit)

Purpose of this code: Admins, assignees and users live in THREE MongoDB collections, so they cannot share one login URL. The form tries admin only when the username is exactly 'admin'. Everyone else is tried as assignee first, then as a user (builder/contractor/user). Passwords are checked with bcrypt on the server, never in the browser.

Code that stores the session

```
localStorage.setItem("duton_access_token", data.token)
localStorage.setItem("duton_token_type", "bearer")
localStorage.setItem("duton_username", trimmedUsername)
localStorage.setItem("duton_user_type", userType) // admin | assignee | user | builder | contractor

router.push("/dashboard")
```

Same file, success branch. Later API calls read duton_access_token.

Purpose of this code: There is no cookie session and no Next.js middleware. The JWT is the entire login. utils/api.js attaches Authorization: Bearer <token> on later requests. lib/user-context.js reads these keys (and the JWT payload) to hide/show admin tabs.

Code that issues the token (backend)

```
export function generateUserToken(user) {
  return jwt.sign(
    {
      id: userId,
      username: user.username,
      email: user.email,
      role: user.role || "user",
    },
    JWT_SECRET,
    { expiresIn: JWT_EXPIRES_IN } // default 24h
  );
}
```

File: duton-backend/src/middleware/auth.js

Purpose of this code: The server signs the token with JWT_SECRET from .env. Protected routes call verifyUserToken or verifyAdminToken, which reject missing/expired tokens with HTTP 401. The browser can decode the payload but cannot forge a valid signature.

10. Procedure F — Dashboard sensor list

Step F.1. After login you land on /dashboard. Wait for sensor cards or table to fill. Optional: switch Grid / Table (admin also Map / Anomalies).

Expected result: Each card shows site name, PM values if live, and Online / Delayed / Offline.

```
export default function DashboardPage() {
  return (
    <div>
      <VideoBackground />
      {!isAdmin && <AmcAlertBanner />}
      <DashboardHeader />
      <main>
        <DashboardContent />
      </main>
      {isAdmin && <CalibrationSetupDialog />}
    </div>
  )
}
```

File: duton-frontend/src/app/dashboard/page.jsx

Purpose of this code: This page is the hub. Header: logo, theme, raise-ticket, logout. AmcAlertBanner is the warranty marquee for builder/contractor. DashboardContent is where sensors are fetched and views/tabs are chosen. The calibration button is admin-only.

How sensors are loaded

```
export const fetchUserSensors = async () => {
  const userType = window.localStorage.getItem("duton_user_type")
  if (userType === "admin") {
    const response = await fetch(`${getTicketApiBaseUrl()}/admin/sensors`, {
      headers: getAuthHeaders(),
    })
    // ... map payload to dashboard shape
  }
  // other roles: GET /api/sensors/user-sensors
}
```

File: duton-frontend/src/utlis/api.js

Purpose of this code: Admin needs every device, so the client calls GET /api/admin/sensors. A normal user only receives sensors from user_sensors (or sites for builder/contractor). getAuthHeaders() adds the Bearer token from Procedure E. Without it this call returns 401.

How live numbers refresh

```
const intervalId = setInterval(refreshAllSensorReadings, 30_000)
// refreshAllSensorReadings → fetchAllLatestReadings(ids)
// GET /api/sensors/latest-all?sensor_ids=...
```

File: duton-frontend/src/components/dashboard/dashboard-content.jsx

Purpose of this code: The dashboard does not use WebSockets. Every 30 seconds it asks Express for the latest sample of every visible sensor. Express may hit Florosense and write sensor_readings. The UI then applies calibration and recalculates status.

How Online / Delayed / Offline is decided

```
// Online: last update within 30 minutes
if (hasCurrentData && minutesSinceLastUpdate <= 30) { /* Online */ }

// Delayed: has data but older than 30 minutes
if (hasCurrentData && minutesSinceLastUpdate > 30) { /* Delayed */ }
```

File: dashboard-content.jsx function calculateSensorStatus

Purpose of this code: This 30-minute rule is UI-only. The backend alert monitor treats offline as no reading for 24 hours. If a card looks Offline but no email was sent, check which threshold you are looking at.

Step F.2. Click a sensor card.

Expected result: Browser goes to /dashboard/{sensor.id} (the monitor page, Procedure G).

11. Procedure G — Open one live sensor

Step G.1. On /dashboard/{id} confirm PM2.5, PM10, temperature and humidity cards update about every 30 seconds.
Expected result: Charts, map, documents and diagnostics sections render below the cards.

```
export const fetchLatestReading = async (sensorIdentifier) => {
  const response = await fetch(
    `${baseUrl}/sensors/latest/${encodeURIComponent(sensorIdentifier)}`,
    { headers: getAuthHeaders() }
  )
  const rawData = payload?.data || payload
  return {
    pm2_5: rawData.pm2_5 ?? rawData.pms_2_5 ?? null,
    pm10_0: rawData.pm10_0 ?? rawData.pms_10 ?? null,
    temperature: rawData.ambient_temperature ?? rawData.temperature ?? null,
    humidity: rawData.relative_humidity ?? rawData.humidity ?? null,
    timestamp: rawData.timestamp ?? null,
  }
}
```

File: duton-frontend/src/utls/api.js — called from MonitoringDashboard

Purpose of this code: Field names from Florosense and from MongoDB are not always the same (pm2_5 vs pms_2_5). This function normalises them so RealTimeDataCards can always read pm2_5 / pm10_0 / temperature / humidity.

Calibration applied on the numbers you see

```
export const applyCalibration = (pmRaw, type = "pm25", sensorId = null) => {
  const config = getCalibrationValuesSync(sensorId)[type]
  const k0 = config.k0 ?? 0
  const k1 = config.k1 ?? 1
  const variation = getVariation(config.variationMin, config.variationMax)
  const pmCorrected = k0 + k1 * pmRaw + variation
  if (pmCorrected < 0) return 0
  return Math.round(pmCorrected * 100) / 100
}
```

File: *duton-frontend/src/utils/calibration.js*

Purpose of this code: Raw device PM can differ from the lab-corrected value the client should see. k0 is offset, k1 is slope, variation is a small random band. Results below zero are clamped. Values are cached in localStorage as `duton_pm_calibration_{sensorId}`.

12. Procedure H — Raise and work a support ticket

Step H.1. On any dashboard page, use Raise ticket in the header. Choose sensor, issue type, description, optional photo. Submit.

Expected result: A new ticket exists with status Open. Ticket Center at `/dashboard/tickets` lists it.

```
export const createSupportTicket = async (payload) => {
  // POST {TICKET_API}/tickets
  // body: sensor_id, issue_type, description, priority, image...
}

export const ISSUE_TYPES = [
  "Data Quality Deviation",
  "Power Supply Issue",
  "Data not live",
  "Other",
]

export const STATUS_FLOW = ["Open", "In Progress", "Resolved", "Closed"]
```

Files: *src/utils/api.js* and *src/lib/support-tickets.js*

Purpose of this code: `createSupportTicket` is the browser side of `POST /api/tickets`. Issue types and status names are shared constants so the form and the Ticket Center stay in sync. Non-admins are limited on the server: max 4 active tickets, one active per sensor, one active per issue type.

Step H.2. Open `/dashboard/tickets`. Filter by status/priority. Admin may set assignee.

Expected result: List refreshes. Assignee may still show empty until the daily email job runs.

```
export function useSupportTickets(filters = {}, options = {}) {
  // GET /api/tickets with filters
  // also listens for window event duton_support_tickets_refresh
}

export const emitTicketRefresh = () => {
  window.dispatchEvent(new Event(TICKET_EVENT_KEY))
}
```

File: *duton-frontend/src/hooks/use-support-tickets.js*

Purpose of this code: The Ticket Center and the Raise-ticket dialog are different components. After a successful create, `emitTicketRefresh()` tells the list to reload without a full page refresh.

Backend: create and deferred assignment

```
router.post("/", upload.single("image"), async (req, res, next) => { /* create ticket */ })
router.patch("/:ticket_id", patchUploadMiddleware, async (req, res, next) => {
  // setting assignee stores pending_assignee until cron dispatches email
})
router.post("/:ticket_id/replies", async (req, res, next) => { /* thread */ })
```

File: duton-backend/src/routes/tickets.js

Purpose of this code: POST creates the document in support_tickets (images as base64). PATCH can change status or queue an assignee. The 10:00 IST job in ticketAssignmentScheduler.js sends the batch email, writes data/ticket-assignment-log.xlsx, then copies pending_assignee into assignee. 18:00 IST sends reminders for tickets still Open or In Progress.

13. Procedure I — Background jobs (no click required)

These start automatically in Procedure B. You do not run them by hand unless you call an admin trigger endpoint.

Job	When	File	What it does
Sensor collector	Every 10 min	services/sensorReadingCollector.js	For each active sensor, GET Florosense latest and store sensor_readings
Alert monitor	Every 15 min	services/alertMonitor.js	Email builders/contractors if offline 24h or PM abnormal 24h
Assignment batch	10:00 IST	services/ticketAssignmentScheduler.js	Email queued assignees and update Excel log
Ticket reminder	18:00 IST	same scheduler	Remind assignees of still-open tickets

Purpose of this code: If live cards work but history is empty, the collector may be failing (check Florosense key). If tickets assign in the UI but nobody gets mail, check SMTP/SendGrid env vars and the 10:00 IST cron timezone Asia/Kolkata.

14. Procedure J — Stop the system

Step J.1. In the backend terminal press Ctrl+C.

Expected result: Logs: Shutting down gracefully... Database connection closed.

```
const shutdown = async () => {
  stopSensorReadingCollector();
  stopTicketAssignmentScheduler();
  stopAlertMonitor();
  server.close(async () => {
    await database.disconnect();
    process.exit(0);
  });
};
process.on("SIGTERM", shutdown);
process.on("SIGINT", shutdown);
```

File: duton-backend/src/server.js

Purpose of this code: SIGINT is Ctrl+C. SIGTERM is what PM2/Nginx send in production. Stopping jobs first avoids writing to MongoDB after the HTTP server is gone. disconnect() closes the Mongo client cleanly.

Step J.2. In the frontend terminal press Ctrl+C.

Expected result: Next.js dev server exits. Browser tabs on :3000 will fail until Procedure C is run again.

15. Procedure K — Guest magic link (optional)

- Admin on a sensor card generates share access (email + duration).
- Backend: POST /api/sensors/:sensor_id/share-access stores a hashed token and SendGrid emails the link.
- Guest opens /access?t=TOKEN&s=SENSOR_ID.
- Frontend validateMagicLink → POST /api/sensors/validate-magic-link → guest JWT in localStorage.
- Dashboard is limited to that one sensor.

16. Procedure L — AMC check (admin + client)

- Admin: /dashboard → AMC tab → pick client → track sensors, installation date, renewal date.
- Rules (amcService.js): warranty = installation + 1 year; AMC = renewal + 1 year; alert window = 90 days.
- Client (builder/contractor): AmcAlertBanner on /dashboard calls GET /api/amc/my-alert.

17. End-to-end workflow map

Human action	Frontend	Backend	Data
Open site	page.js redirect /login	—	—
Sign in	login-form.jsx	POST /api/admin/assignee/users/login	JWT issued
See devices	DashboardContent + fetchUserSensors	GET /api/admin/sensors or /sensors/user-sensors	sensors + assignments
See live PM	30s poll fetchAllLatestReadings	GET /api/sensors/latest-all	Florosense + sensor_readings
Open one device	/dashboard/[id] MonitoringDashboard	GET /api/sensors/latest/:id	one reading
Raise ticket	SupportTicketActions	POST /api/tickets	support_tickets
Assign owner	Ticket Center PATCH	pending_assignee + 10:00 cron	email + Excel log

18. Code index (open this file when you are lost)

File	Purpose
duton-backend/src/server.js	Process entry: middleware, routes, DB, jobs, listen
duton-backend/src/config.js	Load .env; fail fast if Mongo/Florosense missing
duton-backend/src/middleware/auth.js	Sign and verify JWT
duton-backend/src/services/database.js	All existing MongoDB helpers
duton-backend/src/routes/*.js	URL map for each feature
duton-backend/src/controllers/sensorController.js	Live readings, documents, magic links
duton-frontend/src/app/layout.js	HTML shell, theme, toasts
duton-frontend/src/app/page.js	Redirect / → /login
duton-frontend/src/components/auth/login-form.jsx	Login cascade + localStorage
duton-frontend/src/lib/user-context.js	Read role flags from token
duton-frontend/src/utlis/api.js	Almost every HTTP call from the UI
duton-frontend/src/utlis/calibration.js	PM correction on the client
duton-frontend/src/components/dashboard/dashboard-content.jsx	Hub: fetch, views, admin tabs

File	Purpose
duton-frontend/src/components/monitoring/monitoring-dashboard.jsx	One-sensor live page
duton-frontend/next.config.mjs	Proxy /api/* to Express; calibration-proxy to Florosense

19. Quick restart checklist

- Terminal 1: cd duton-backend && npm run dev → wait for Mongo connected.
- Terminal 2: cd duton-frontend && npm run dev → wait for :3000.
- Browser: http://127.0.0.1:8001/health then http://localhost:3000
- Log in → /dashboard → click a sensor → confirm PM cards update.
- Optional: /dashboard/tickets to confirm ticket API.

To regenerate this PDF after you change the SOP script: py -3 docs/scripts/generate_sop_pdf.py